

Video Processing Subsystem v2.0

Product Guide

Vivado Design Suite

PG231 October 5, 2016

Table of Contents

IP Facts

Chapter 1: Overview

Introduction	5
Feature Summary.....	5
Applications	6
Licensing and Ordering Information.....	6

Chapter 2: Product Specification

Standards	8
Performance.....	8
Resource Utilization.....	9
Port Descriptions	10

Chapter 3: Designing with the Core

General Design Guidelines	15
Clocking.....	33
Resets	34

Chapter 4: Design Flow Steps

Customizing and Generating the Core	35
Constraining the Core	45
Simulation	46
Synthesis and Implementation	46

Chapter 5: Detailed Example Design

Full Fledged and Scaler Only Video Processing Design	47
Deinterlacing, Color Space Conversion, or Chroma Resampling Only Video Processing Design ..	48

Appendix A: Migrating and Upgrading

Upgrading in the Vivado Design Suite	52
--	----

Appendix B: Debugging

Finding Help on Xilinx.com	53
----------------------------------	----

Debug Tools	54
Simulation Debug.....	55
Hardware Debug	55
Interface Debug	56

Appendix C: Application Software Development

Driver	57
Dependencies.....	57
Architecture	57
Usage	59

Appendix D: Additional Resources and Legal Notices

Xilinx Resources	61
References	61
Revision History	62
Please Read: Important Legal Notices	62

Introduction

The Video Processing Subsystem is a collection of video processing IP subcores, bundled together in hardware and software, abstracting the video processing pipe. It provides the end-user with an out of the box ready to use video processing core, without having to learn about the underlying complexities. The Video Processing Subsystem enables streamlined integration of various processing blocks including (but not limited to) scaling, deinterlacing, color space conversion and correction, chroma resampling, and frame rate conversion.

Features

- One, two or four pixel-wide AXI4-Stream video interface
- Video resolution support up to UHD at 60 fps
- Run-time color space support for RGB, YUV 4:4:4, YUV 4:2:2, YUV 4:2:0
- 8, 10, 12, and 16 bits per component support
- Deinterlacing
- Scaling
- Color space conversion and correction
- Chroma resampling between YUV 4:4:4, YUV 4:2:2, YUV 4:2:0
- Frame rate conversion using dropped/repeated frames

LogiCORE™ IP Facts Table	
Core Specifics	
Supported Device Family ⁽¹⁾	UltraScale+™ Families, UltraScale Architecture, Zynq®-7000, 7 Series
Supported User Interfaces	AXI4-Lite, AXI4-Stream, AXI-MM
Resources	See Table 2-1
Provided with Core	
Design Files	Encrypted HLS C
Example Design	Verilog
Test Bench	Not Provided
Constraints File	XDC
Simulation Model	Not Provided
Supported S/W Driver ⁽²⁾	Standalone
Tested Design Flows ⁽³⁾	
Design Entry	Vivado® Design Suite
Simulation	For supported simulators, see the Xilinx Design Tools: Release Notes Guide .
Synthesis	Vivado Design Suite
Support	
Provided by Xilinx at the Xilinx Support web page	

Notes:

1. For a complete list of supported devices, see the Vivado IP catalog.
2. Standalone driver details can be found in the SDK directory (`<install_directory>/doc/usenglish/xilinx_drivers.htm`). Linux OS and driver support information is available from the [Xilinx Wiki page](#).
3. For the supported versions of the tools, see the [Xilinx Design Tools: Release Notes Guide](#).

Overview

Introduction

The Video Processing Subsystem enables streamlined integration of various processing blocks including (but not limited to) scaling, deinterlacing, color space conversion and correction, chroma resampling, and frame rate conversion.

Feature Summary

The Video Processing Subsystem has the following features:

- One, two, or four pixel-wide video interface
- Run-time color space support for RGB, YUV 4:4:4, YUV 4:2:2, YUV 4:2:0
- 8, 10, 12, and 16 bits per component support
- Deinterlacing
- Scaling
- Color space conversion and correction
- Chroma resampling between YUV 4:4:4, YUV 4:2:2, YUV 4:2:0
- Frame rate conversion
- Supports resolutions up to 4096 x 2160

The Video Processing Subsystem is a hierarchical IP that bundles a collection of video processing IP subcores and outputs them as a single IP. The Video Processing Subsystem has design time configurability in terms of performance and quality. You can configure the subsystem IP through one single graphical user interface. A preview of this GUI is shown in [Figure 4-1](#).

All video processing IP subcores have been developed using Vivado HLS.

Applications

- Color space (RGB/YUV) and format (YUV 4:4:4/4:2:2/4:2:0) conversion.
- Scale up and down up to 4k2k at 60 Hz.
- Zoom mode, where in a user defined window, the input stream is scaled to panel resolution.
- Picture-In-Picture mode where in the input stream is scaled down to a user defined window size and displayed at the user defined co-ordinates on the panel.
- Ability to paint the PIP background to a defined color.
- Interlaced to progressive conversion.
- Frame rate conversion.
 - Drop frames if input rate > output rate.
 - Repeat frames if output rate < input rate.

Licensing and Ordering Information

License Checkers

If the IP requires a license key, the key must be verified. The Vivado® design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with error. License checkpoints are enforced by the following tools:

- Vivado synthesis
- Vivado implementation
- write_bitstream (Tcl command)



IMPORTANT: IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.

License Type

This Xilinx LogiCORE™ IP module is provided under the terms of the [Xilinx Core License Agreement](#). The module is shipped as part of the Vivado® Design Suite. For full access to all core functionalities in simulation and in hardware, you must purchase a license for the core. There is no evaluation version of the core. Contact your [local Xilinx sales representative](#) for information about pricing and availability.

For more information, visit the Video Processing Subsystem product web page.

www.xilinx.com/products/intellectual-property/video-processing-subsystem.html

Information about other Xilinx LogiCORE IP modules is available at the [Xilinx Intellectual Property](#) page. For information on pricing and availability of other Xilinx LogiCORE IP modules and tools, contact your [local Xilinx sales representative](#).

Product Specification

Standards

The Video Processing Subsystem core is compliant with the AXI4-Stream Video Protocol and AXI4-Lite interconnect standards. Refer to the *Video IP: AXI Feature Adoption* section of the *Vivado AXI Reference Guide* (UG1037) [Ref 8] for additional information.

Performance

Maximum Frequencies

The following are typical clock frequencies for the target devices:

- Virtex®-7, Virtex UltraScale™, and Virtex UltraScale+ devices with –2 speed grade or higher: 300 MHz
- Kintex®-7, Kintex UltraScale™, and Kintex UltraScale+ devices with –2 speed grade or higher: 300 MHz
- Artix®-7 devices with –2 speed grade or higher: 150 MHz

The maximum achievable clock frequency can vary. The maximum achievable clock frequency and all resource counts can be affected by other tool options, additional logic in the device, using a different version of Xilinx® tools, and other factors.

Latency

The latency of the Video Processing Subsystem depends on the configuration of the core. Generally, the latency is on the order of several lines. For example, a vertical scaler with a 6-tap polyphase filter operating on RGB video introduces 4 video lines of delay.

In the Full Fledged configuration with the use of a DMA, the latency is 1 full frame time plus several lines because the video DMA engine is used in the data flow and programmed to read one frame buffer behind the write frame buffer location.

In interlaced video, 1 field additional delay is incurred by the deinterlacer algorithm.

Resource Utilization

Table 2-1 shows representative resource utilization for several configurations of this IP core. In Table 2-1, each row describes a test case. The columns are divided into test parameters and results. The test parameters describe the core configuration. Any configuration parameters that are not listed have their default values.

The default Vivado Design Suite settings were used. You may be able to optimize on these numbers using different settings. Because surrounding circuitry affects placement and timing, no guarantee can be given that these figures are repeatable in a larger design.

All configurations (except where noted), are configured for 2 Samples per Clock, 10 bit Data Width, and a frame size of 3840 x 2160.

Table 2-1: Video Processing Subsystem Resource Utilization

Configuration	LUTs	FFs	LUT-FF Pairs	DSP48s	36k BRAMs	18k BRAMs
Full Fledged	28355	37408	13495	126	115	20
Full Fledged deinterlacing disabled DMA excluded	17828	24312	8642	122	81	0
Scaler Only RGB/YUV444/422/420 support color space conversion enabled	7478	11244	4338	88	57	1
Scaler Only RGB/YUV 444 support only	5312	7935	3053	72	42	1
Deinterlacing Only 1 sample per clock 1920x1080 frame size	4331	5024	2183	1	30	5
Color Space Conversion Only RGB/YUV 444 support only demo window disabled	1492	1540	721	18	0	0
420-422 Chroma Resampling Only	1310	1919	788	8	13	0
422-444 Chroma Resampling Only	1344	2262	824	16	0	0

Notes:

1. These numbers were generated using Vivado Design Suite 2016.3 with Speedfile PRODUCTION 1.12 2014-09-11 on Device xc7k325ffg900-2.

Port Descriptions

Figure 2-1 shows the Video Processing Subsystem diagram in its Full Fledged configuration. The IP has four AXI interfaces:

- AXI4-Stream streaming video input (`s_axis`)
- AXI4-Stream streaming video output (`m_axis`)
- AXI-MM memory mapped interface (`m_axi_mm`)
- AXI-Lite control interface (`s_axi_ctrl`).



Figure 2-1: Full Fledged Video Processing Subsystem

The AXI Streaming, AXI Memory-Mapped, and AXI Lite interfaces can be run at their own clock rate, therefore, three separate clock interfaces are provided named `aclk_axis`, `aclk_axi_mm`, and `aclk_ctrl`, respectively. The `aresetn_ctrl` signal is the reset signal of the IP, and `aresetn_io_axis` is an outgoing signal that can be used to hold IPs in reset when the Video Processing Subsystem is not ready to consume data on the streaming input. Finally, `deint_field_id` signal indicates field polarity in case of interlaced operation.

AXI4-Stream Video

The Video Processing Subsystem has AXI4-Stream video input and output interfaces named `s_axis` and `m_axis`, respectively. These interfaces follow the interface specification as defined in the *Video IP* chapter of the *Vivado AXI Reference Guide* (UG1037) [Ref 8]. The video AXI4-Stream interface can be single, dual, or quad pixels per clock and can support 8,

10, 12, or 16 bits per component. For example, the pixel mapping per color format and bus signals for 10 bits per component are shown in Table 2-2 through Table 2-6.

Table 2-2: Dual Pixels per Clock, 10 Bits per Component Mapping for RGB

63:60	59:50	49:40	39:30	29:20	19:10	9:0
zero padding	R1	B1	G1	R0	B0	G0

Table 2-3: Dual Pixels per Clock, 10 Bits per Component Mapping for YUV 4:4:4

63:60	59:50	49:40	39:30	29:20	19:10	9:0
zero padding	V1	U1	Y1	V0	U0	Y0

Table 2-4: Dual Pixels per Clock, 10 Bits per Component Mapping for YUV 4:2:2

63:60	59:50	49:40	39:30	29:20	19:10	9:0
zero padding	zero padding	zero padding	V0	Y1	U0	Y0

Table 2-5: Dual Pixels per Clock, 10 Bits per Component Mapping for YUV 4:2:0, for Even Lines

63:60	59:50	49:40	39:30	29:20	19:10	9:0
zero padding	zero padding	zero padding	V0	Y1	U0	Y0

Table 2-6: Dual Pixels per Clock, 10 Bits per Component Mapping for YUV 4:2:0, for Odd Lines

63:60	59:50	49:40	39:30	29:20	19:10	9:0
zero padding	zero padding	zero padding	zero padding	Y1	zero padding	Y0

This IP always generates three video components even if the video format is set to be YUV 4:2:0 or YUV 4:2:2 at run-time. The unused components can be set to zero.

Table 2-7 shows the interface signals for input and output AXI4-Stream video streaming interfaces.

Table 2-7: AXI4 Streaming Interface Signals

Name	Direction	Width	Description
s_axis_tdata	In	$\text{floor}(((3 \times \text{bits_per_component} \times \text{pixels_per_clock}) + 7) / 8) \times 8$	Input Data
s_axis_tready	Out	1	Input Ready
s_axis_tvalid	In	1	Input Valid
s_axis_tid	In	1	Input data stream identifier
s_axis_tdest	In	1	Input data routing identifier
s_axis_tkeep	In	$(\text{s_axis_video_tdata width})/8$	Input byte qualifier that indicates whether the content of the associated byte of TDATA is processed as part of the data stream
s_axis_tlast	In	1	Input End of Line

Table 2-7: AXI4 Streaming Interface Signals (Cont'd)

Name	Direction	Width	Description
s_axis_tstrb	In	$(s_axis_video_tdata\ width)/8$	Input byte qualifier that indicates whether the content of the associated byte of TDATA is processed as a data byte or a position byte
s_axis_tuser	In	1	Input Start of frame
m_axis_tdata	Out	$\text{floor}(((3 \times \text{bits_per_component} \times \text{pixels_per_clock}) + 7) / 8) \times 8$	Output Data
m_axis_tdest	Out	1	Output data routing identifier
m_axis_tid	Out	1	Output data stream identifier
m_axis_tkeep	Out	$(m_axis_video_tdata\ width)/8$	Output byte qualifier that indicates whether the content of the associated byte of TDATA is processed as part of the data stream
m_axis_tlast	Out	1	Output End of Line
m_axis_tready	In	1	Output Ready
m_axis_tstrb	Out	$(m_axis_video_tdata\ width)/8$	Output byte qualifier that indicates whether the content of the associated byte of TDATA is processed as a data byte or a position byte
m_axis_tuser	Out	1	Output Start of frame
m_axis_tvalid	Out	1	Output Valid

Both video streaming interfaces run at the video stream clock speed `aclk_axis`.

AXI-MM Memory-Mapped Interface

The video DMA read and write ports and the deinterlacer read and write ports are concentrated by an AXI-MM cross-bar interconnect such that there is only one AXI-MM interface on the boundary of the subsystem. The AXI-MM interface runs on the `aclk_axi_mm` clock domain. The signals follow the specification as defined in the *AXI Reference Guide* [Ref 8].

The AXI-MM Memory-Mapped interface is only present with the Full Fledged functionality and Deinterlacer Only configurations. The other configurations do not require access to external memory.

AXI-Lite Control Interface

Table 2-8 shows the AXI-Lite control interface signals. This interface runs at the `aclk_ctrl` clock. Control of the video processing pipe is only supported through the Video Processing Subsystem driver.

Table 2-8: AXI Lite Control Interface

Name	Direction	Width	Description
s_axi_ctrl_awaddr	In	20	Write address
s_axi_ctrl_awprot	In	3	Write address protection
s_axi_ctrl_awvalid	In	1	Write address valid
s_axi_ctrl_awready	Out	1	Write address ready
s_axi_ctrl_wdata	In	32	Write data
s_axi_ctrl_wstrb	In	4	Write data strobe
s_axi_ctrl_wvalid	In	1	Write data valid
s_axi_ctrl_wready	Out	1	Write data ready
s_axi_ctrl_bresp	Out	2	Write response
s_axi_ctrl_bvalid	Out	1	Write response valid
s_axi_ctrl_bready	In	1	Write response ready
s_axi_ctrl_araddr	In	20	Read address
s_axi_ctrl_arprot	In	3	Read address protection
s_axi_ctrl_arvalid	In	1	Read address valid
s_axi_ctrl_aready	Out	1	Read address ready
s_axi_ctrl_rdata	Out	32	Read data
s_axi_ctrl_rresp	Out	2	Read data response
s_axi_ctrl_rvalid	Out	1	Read data valid
s_axi_ctrl_rready	In	1	Read data ready

Clocks and Resets

Table 2-9 provides an overview of the clocks and resets. See section [Clocking in Chapter 3](#) for more information.

Table 2-9: Clocks and Resets

Name	Direction	Width	Description
Clocks			
aclk_axis	In	1	Clock at which AXI4-Stream input and output are running.
aclk_ctrl	In	1	AXI-Lite clock for CPU control interface.
aclk_axi_mm	In	1	Clock at which AXI-MM interface is running.

Table 2-9: Clocks and Resets (Cont'd)

Name	Direction	Width	Description
Resets			
aresetn_ctrl	In	1	Reset, associated with aclk_ctrl (active Low). The aresetn_ctrl signal resets the entire IP including the data path and AXI4-Lite registers.
aresetn_io_axis	Out	1	Used to hold upstream logic in reset while the Video Processing Subsystem is not ready to consume data on streaming input (active Low).

Field Polarity

The `deint_field_id` signal indicates the polarity of the incoming field when the input video is interlaced. This signal is only used by the deinterlacer with interlaced data. This signal is ignored for progressive video inputs.

Table 2-10: Field Polarity

Name	Direction	Width	Description
deint_field_id	In	1	Field polarity, odd is low, high is even

Designing with the Core

This chapter includes guidelines and additional information to facilitate designing with the core.

General Design Guidelines

The Video Processing Subsystem is a collection of individual subcore IPs packaged as a hierarchical IP and configured through one single graphical user interface (GUI). The Subsystem can perform the following functions: deinterlacing, scaling, frame rate conversion, color space conversion and correction, and chroma resampling.

[Appendix C, Application Software Development](#) describes how to integrate the associated Video Processing Subsystem API into a software application.

Deinterlacing

The Video Deinterlacer converts live incoming interlaced video streams into progressive video streams. Interlaced images may have temporal motion between the two fields that comprise an interlaced frame. The conversion to a progressive format recombines these two fields into one single progressive scan frame. The combining of interlaced video streams results in unsightly motion artifacts in the progressive output image. For this reason, the Video Deinterlacer can be configured to use three field buffers and produce progressive frames based on a combination of spatial and temporal processing.

The following deinterlacing algorithms are supported:

Bob (Line Doubling): The Bob algorithm takes the lines of each interlaced field (consisting of only even or odd lines of a progressive frame) and duplicates them, filling the entire frame. Line doubling prevents "combing" artifacts but causes a noticeable reduction in video quality. This is noticeable mostly on stationary or slowly moving objects since they seem to bob up and down. This algorithm does not require external frame buffers.

Weave: Weaving is done by assembling 2 consecutive fields together. This performs well when the image hasn't changed between fields, but any move will cause "combing" artifacts - when the pixels in one frame do not align with the pixels in the next frame, forming a jagged edge. Weave algorithm is a motion adaptive method and requires external frame buffers.

Vertical Temporal Linear Interpolation (VTLin): VTLin generates intermediate lines by linear interpolation of its 2 neighboring lines from the current field and 3 reference lines from the previous field. The generated lines are added to lines of the current field, forming the entire frame. VTLin performs better than Bob or Weaving for moving objects, but interpolation results in slight image blurring. VTLin is a motion adaptive algorithm and requires an external frame buffer.

Vertical Temporal Median (VTMed): VTMed makes the generated lines by taking the median of its 2 neighboring lines from the current field and the reference line from the previous field. The generated lines are added to lines of the current field, forming the entire frame. VTMed is a motion adaptive algorithm.

Median: The Median algorithm takes the median of the result from VTLin and VTMed in 2 consecutive fields and the reference line in the next field as the generated lines. The output frame is consisted of generated lines and reference lines of current field. Median is a motion adaptive method and requires an external frame buffer.

Bilinear Interpolation: This algorithm is done by performing bilinear interpolation on the current interlaced field. It fills generated lines by taking the average of 2 neighboring reference lines. The Bilinear Interpolation algorithm is not motion adaptive. It works well for stationary frames but unsatisfactorily for fast moving objects.

Features

The Video Deinterlacer is a low-cost basic deinterlacer. The following list is a summary of the supported features:

- Support for six deinterlacing algorithms. The Bob and Bilinear Interpolation algorithms do not require field buffers. Weave, Vertical Temporal Linear Interpolation, Vertical Temporal Median, and Median algorithms all require field buffers.
- Support for RGB, YUV 4:4:4, YUV 4:2:2, and YUV 4:2:0
- 8, 10, 12, or 16 bits per component.

Scaling

Video scaling is the process of converting an input color image of dimensions X_{in} pixels by Y_{in} lines to an output color image of dimensions X_{out} pixels by Y_{out} lines. The IP converts a specified rectangular area of an input digital video image from the original sampling grid to a desired target sampling grid.

The input image must be provided in raster scan format (left to right and top to bottom). The valid outputs are also given in this order.

Video scaling is a form of 2-D filter operation which can be approximated with the equation shown in [Equation 3-1](#).

Equation 3-1

$$Pix_{out}(x,y) = \sum_{HTaps-1}^{i=0} \sum_{VTaps-1}^{j=0} Pix_{in}[x - (HTaps / 2) + i, y - (VTaps / 2) + j] \times Coef(i,j)$$

In this equation, x and y are discrete locations on a common sampling grid; $Pix_{out}(x, y)$ is an output pixel that is being generated at location (x, y) ; $Pix_{in}(x, y)$ is an input pixel being used as part of the input scaler aperture; $Coef(i, j)$ is an array of coefficients that depend upon the application; and $HTaps$ and $VTaps$ are the number of horizontal and vertical taps in the filter, respectively.

The coefficients in this equation represent weights applied to the set of input samples chosen to contribute to one output pixel, according to the scaling ratio.

Features

The Scaler comes in three different quality levels each at different levels of resource usage.

- Bilinear scaling is the cheapest implementation of the Scaler that uses bilinear interpolation to calculate pixels. Bilinear interpolation produces a greater number of interpolation artifacts (such as aliasing, blurring, and edge halos) than more computationally demanding techniques such as bicubic interpolation.
- Bicubic scaling is a more demanding compared to bilinear scaling, and produces smoother pictures with less artifacts. Compared to bilinear interpolation, which only takes 2×2 pixels into account, bicubic interpolation considers a 4×4 pixel area.
- The polyphase concept is explained in [Polyphase Scaling](#). The picture quality (and resource usage) of a polyphase scaler depends largely on the number of filter taps used and number of filter phases used. Polyphase scaling offers the highest quality but also has the highest resource utilization.

The following list is a summary of the supported features:

- Polyphase, bicubic, or bilinear scaling modes
- 6, 8, 10, or 12-tap 64 phase polyphase scaling
- Support for and conversion between RGB, YUV 4:4:4, YUV 4:2:2, and YUV 4:2:0
- 8, 10, 12, or 16 bits per video component

Polyphase Scaling

For scaling, the input and output sampling grids are assumed to be different. To express a discrete output pixel in terms of input pixels, it is necessary to know or estimate the location of the output pixel relative to the closest input pixels when superimposing the output sampling grid upon the input sampling grid for the equivalent 2-D space. With this

knowledge, the algorithm approximates the output pixel value by using a filter with coefficients weighted accordingly. Filter taps are consecutive data-points drawn from the input image.

As an example, Figure 3-1 shows a desired 5x5 output grid ("O") superimposed upon an original 6x6 input grid ("X"), occupying common space. In this case, estimating for output position $(x, y) = (1, 1)$, shows the input and output pixels to be co-located. You can weigh the coefficients to reflect no bias in either direction, and can even select a unity coefficient set. Output location $(2, 2)$ is offset from the input grid in both vertical and horizontal dimensions. Coefficients can be chosen to reflect this, most likely showing some bias towards input pixel $(2, 2)$, etc. Filter characteristics can be built into the filter coefficients by appropriately applying anti-aliasing low-pass filters.

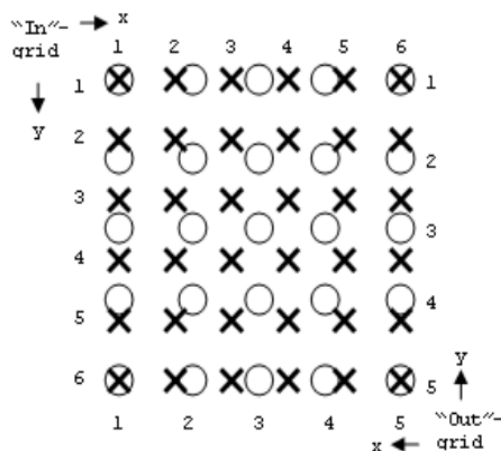


Figure 3-1: 5x5 Output Grid ("O") Super-imposed over 6x6 Input Grid ("X")

The space between two consecutive input pixels in each dimension is conceptually partitioned into a number of bins or phases. The location of any arbitrary output pixel always falls into one of these bins, thus defining the phase of coefficients used. The filter architecture should be able to accept any of the different phases of coefficients, changing phase on a sample-by-sample basis.

A single dimension is shown in Figure 3-2. As illustrated in this figure, the five output pixels shown from left to right could have the phases 0, 1, 2, 3, 0.

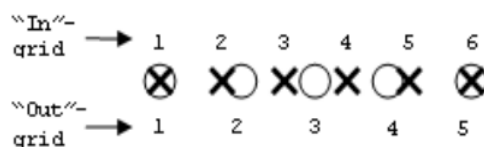


Figure 3-2: Super-imposed Grids for 1 Dimension

The examples in Figure 3-1 and Figure 3-2 show a conversion where the ratio $X_{in}/X_{out} = Y_{in}/Y_{out} = 5/4$. This ratio is known as the Scaling Factor, or SF. The horizontal and vertical Scaling Factors can be different. A typical example is drawn from the broadcast industry,

where some footage can be shot using 720p (1280 x 720), but the cable operator needs to deliver it as per the broadcast standard 1080p (1920 x 1080). The SF becomes 2/3 in both H and V dimensions.

Typically, when $X_{in} > X_{out}$, this conversion is known as horizontal down-scaling ($SF > 1$). When $X_{in} < X_{out}$, it is known as horizontal up-scaling ($SF < 1$).

The set of coefficients constitute filter banks in a polyphase filter whose frequency response is determined by the amount of scaling applied to the input samples. The phases of the filter represent subfilters for the set of samples in the final scaled result.

The number of coefficients and their values are dependent upon the required low-pass, anti-alias response of the scaling filter; for example, smaller scaling ratios require lower passbands and more coefficients. Filter design programs based on the Lanczos algorithm are suitable for coefficient generation. Moreover, MATLAB® product fdatool/fvtool can be used to provide a wider filter design toolset.

A general guideline is to use 4 taps per number of scaling ratio for scaling down to get good quality. The following are some recommendations for how many taps to use:

Upscale	6 taps
Down scale to 1.5	6 taps
Down scale $> 1.5 \leq 2.5$	8 taps
Down scale $> 2.5 \leq 3.5$	10 taps
Down scale > 3.5	12 taps

A direct implementation of Equation 1 suggests that a filter with $VTaps \times HTaps$ multiply operations per output are required. However, the Xilinx® Video Scaler supports only separable filters, which completes an approximation of the 2-D operation using two 1-D stages in sequence - a vertical filter (V-filter) stage and a horizontal filter (H-filter) stage. The intermediate results of the first stage are fed sequentially to the second stage.

The vertical filter stage filters only in the vertical domain, for each incrementing horizontal raster scan position x , creating an intermediate result described as $VPix$ (Equation 3-2).

$$VPix_{int}[x, y] = \sum_{i=0}^{VTaps-1} Pix_{in}[x, y - (VTaps/2) + i] \times Vcoef[i] \quad \text{Equation 3-2}$$

The output result of the vertical component of the scaler filter is input into the horizontal filter with the appropriate rounding applied. The separation means this can be reduced to the shown $VTaps$ and $HTaps$ multiply operations, saving FPGA resources (Equation 3-3).

$$Pix_{out}[x, y] = \sum_{i=0}^{HTaps-1} VPix_{int}[x - (HTaps/2) + i, y] \times Hcoef[i] \quad \text{Equation 3-3}$$

Notice that the differences between the Bilinear, Bicubic, and Polyphase architectures are not only marked by a difference in coefficients but with the implementation of optimized architectures for Bilinear and Bicubic scaling.

AXI Video Direct Memory Access Engine

Many video applications require frame buffers to handle frame rate changes or changes to the image dimensions (scaling or cropping). The Video DMA engine, which uses the Xilinx AXI Video Direct Memory Access IP, is designed to allow for efficient high-bandwidth access between AXI4-Stream video interface and AXI4-MM interface. See the *AXI Video Direct Memory Access Product Guide* [Ref 9] for more information.

Features

The Video DMA engine within the Full Fledged configuration of the Video Processing Subsystem IP is used to enable frame rate conversion and also more advanced scaling use cases like crop and zoom and scale to picture in picture.

Frame Rate Conversion

Frame rate conversion is implemented by dropping or repeating frames. The DMA engine keeps track of 4 frame buffers that are being written to in a cyclic fashion. The read portion of the video data flow remains exactly 1 frame behind the write pointer. In case the incoming frame rate is higher than the outgoing frame rate, the write pointer advances more frequently than the read pointer, meaning that frames are skipped. Similarly, the read pointer is always one frame behind the write pointer even in case the outgoing frame rate is higher than the incoming frame rate. In this case, frames are repeated.

Crop and Zoom

Another use of the video DMA engine is to enable advanced scaling use cases like crop and zoom, and scale to picture in picture. Without the use of memory, only basic scaling can be performed from one resolution to another resolution, as shown in [Figure 3-3](#).

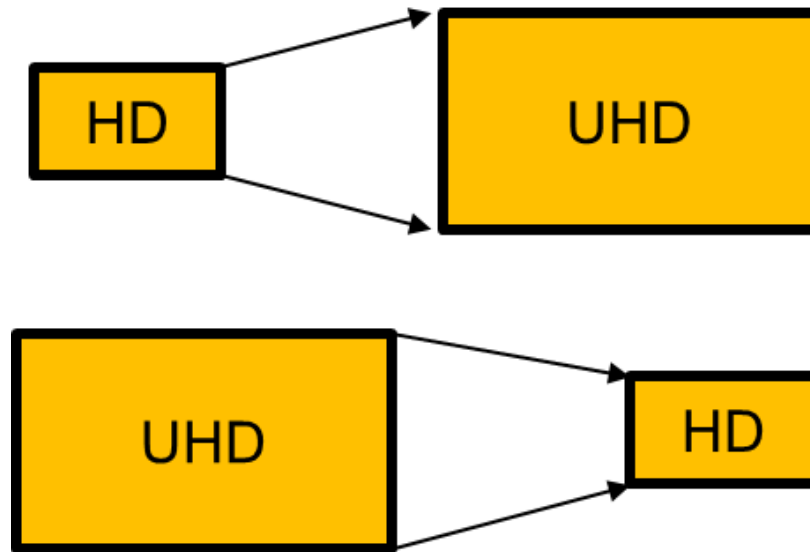


Figure 3-3: Basic Scaling

With the video DMA engine in the video path, it is possible to enable a crop and zoom feature as shown in Figure 3-4 by cropping out of memory.

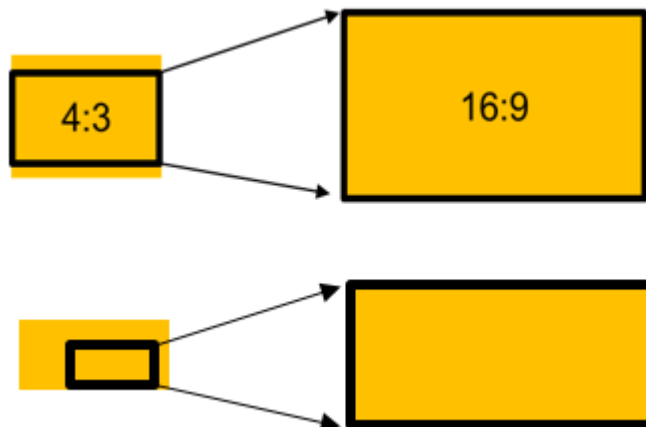


Figure 3-4: Crop and Zoom

Picture in Picture

Alternatively, it is also possible to enable a picture in picture feature as shown in Figure 3-5.

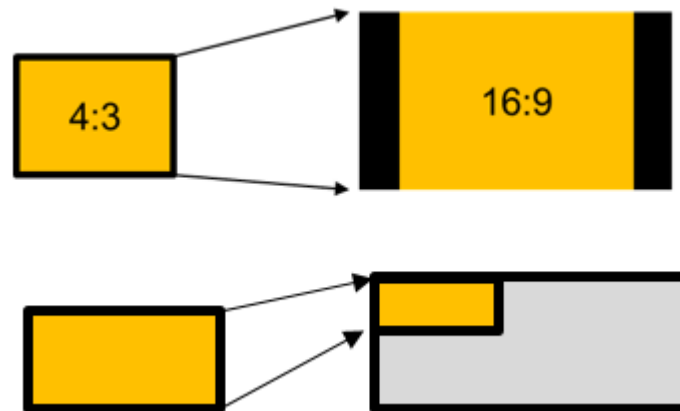


Figure 3-5: Picture in Picture

Note that there is no data re-alignment engine implemented, therefore, memory access is aligned to the granularity of the bus. This is automatically taken care of by the driver.

Memory Requirement

DDR memory is used to store video frame buffers for full configuration mode. The subsystem uses 5 frame buffers for progressive input and 3 field buffers for interlaced input. You can calculate the amount of memory required by the subsystem using the following equations:

$$\begin{aligned}
 &5 * \text{MAX_WIDTH}_p * \text{MAX_HEIGHT}_p * \text{NUM_VIDEO_COMPONENTS} * \text{BytesPerComp} \\
 &+ \\
 &3 * \text{MAX_WIDTH}_i * \text{MAX_HEIGHT}_i * \text{NUM_VIDEO_COMPONENTS} * \text{BytesPerComp}
 \end{aligned}$$

BytesPerComp

- 1 Byte for 8 bit data
- 2 Byte for 10/12/16 bit data

Memory Bandwidth

The Full Fledged configuration has the following memory bandwidth requirements:

The deinterlacer writes 1 field and reads 2 fields. For example, a resolution of 1080i of 8 bit RGB data at 60 Hz:

$$\text{write 1 field} = 1920 \text{ columns} * 540 \text{ rows} * 24\text{bits} * 60 \text{ fps} = 178\text{MBytes/second}$$

read 2 fields = 1920 columns * (2*540) rows * 24bits * 60 fps = 356MBytes/second

The VDMA writes 1 frame and reads 1 frame. For example, a 4K resolution of 8 bit RGB data at 60 Hz:

write 1 frame = 3840 columns * 2160 rows * 24 bits * 60 fps = 1424MBytes/second

read 1 frame = 3840 columns * 2160 rows * 24 bits * 60 fps = 1424MBytes/second

Color Space Conversion and Correction

There are many variations that cause difficulties in accurately reproducing color in imaging systems. These can include:

- Spectral characteristics of the optics (lens, filters)
- Lighting source variations like daylight, fluorescent, or tungsten
- Characteristics of the color filters of the sensor

The Color Space Converter/Correction function provides a method for correcting the image data for these variations. This fundamental block operates on either YUV or RGB data.

As an example, following one of the three color channels through an imaging system from the original light source to the processed image helps understand the functionality of this core.

The blue color channel is a combination of the blue photons from the scene, multiplied by the relative response of the blue filter, multiplied by the relative response of the silicon to blue photons. However, the filter and silicon responses might be quite different from the response of the human eye, so blue to the sensor is quite different from blue to a human being.

This difference can be corrected and made to more closely match the blue that is acceptable to human vision. The Color Space Converter/Correction function multiplies the pixel values by some coefficient to strengthen or weaken it, creating an effective gain. At the same time a mixture of green or red can be added to the blue channel. To express this processing mathematically, the new blue (B_c) is related to the old blue (B), red (R), and green (G) according to:

$$B_c = K1 \times R + K2 \times G + K3 \times B \quad \text{Equation 3-4}$$

where $K1$, $K2$, and $K3$ are the weights for each of the mix of red, green, and blue to the new blue.

Extending this concept, a standard 3 x 3 matrix multiplication can be applied to each of the color channels in parallel simultaneously. This is a matrix operation where the weights define a color-correction matrix. In typical applications, color-correction also contains offset compensation to ensure black [0,0,0] levels are achieved.

$$\begin{bmatrix} R_c \\ G_c \\ B_c \end{bmatrix} = \begin{bmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} O_1 \\ O_2 \\ O_3 \end{bmatrix} \quad \text{Equation 3-5}$$

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{bmatrix} \begin{bmatrix} Y \\ U \\ V \end{bmatrix} + \begin{bmatrix} O_1 \\ O_2 \\ O_3 \end{bmatrix} \quad \text{Equation 3-6}$$

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{bmatrix} \begin{bmatrix} Y \\ U \\ V \end{bmatrix} + \begin{bmatrix} O_1 \\ O_2 \\ O_3 \end{bmatrix} \quad \text{Equation 3-7}$$

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \begin{bmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} O_1 \\ O_2 \\ O_3 \end{bmatrix} \quad \text{Equation 3-8}$$

As shown in the matrix operation, the input pixels are transformed to a set of corrected output pixels. This can be a very useful function configured as a static application; however, the programmability of the coefficients and offset values allows this function to adapt to changing lighting conditions based on a separate control loop.

Features

The Color Space Conversion and Correction function offers a 3 x 3 matrix multiplication for a variety of color correction applications. The coefficient matrix is fully programmable and includes offset compensation, and clipping and clamping of the output is also definable.

The following list is a summary of the supported features:

- User programmable matrix coefficients
- Support for RGB, YUV 4:4:4, and YUV 4:2:2
- 8, 10, 12, or 16 bits per component

- Driver API to set coefficients for converting RGB to YUV, or vice-versa
- Driver API to set/get brightness, contrast, saturation and gain

Color Space Conversion

The primary purpose of this function is to provide color space conversion between the RGB and YUV domains. The fully programmable 3 x 3 matrix with offsets and clipping and clamping allow the support of multiple video standards.

Color Correction

This function provides support for additional color correction within a user defined window in the video frame. You can define a second coefficient matrix to be applied only within a demo window. You also program the size and position of the demo window.

Filter Coefficients and Offsets

The coefficients are presented in 16.12 fixed point format. The 16-bit signed integer values (2's compliment) are equivalent to real numbers in the [-8 .. 8] range.

The offset value has a width of the data width plus 1. It is a signed integer with a range as shown in [Equation 3-9](#).

$$[-2^{\text{Data_Width}}, 2^{\text{Data_Width}-1}] \quad \text{Equation 3-9}$$

Matrix computation outputs are rounded to DATA_WIDTH bits by adding half an output LSB prior to truncation.

Output values greater than the Clip value are replaced with the Clip value. Output values smaller than the Clamp value are replaced with the Clamp value. The Clip and Clamp values have the same width as the data width. They are unsigned integers with a range of [0 .. $2^{\text{Data_Width}-1}$].

Chroma Resampling

The human eye is not as receptive to chrominance (color) detail as luminance (brightness) detail. Using color-space conversion, it is possible to convert RGB into the YUV color space, where Y is Luminance information, and U and V are derived color difference signals. At normal viewing distances, there is no perceptible loss incurred by sampling the color difference signals (U and V) at a lower rate to provide a simple and effective video compression to reduce storage and transmission costs

The Chroma Resampler function converts between chroma sub-sampling formats of 4:4:4, 4:2:2, and 4:2:0. There are a total of six conversions available for the three supported sub-sampling formats. Conversion is achieved using a FIR filter approach. Some conversions require filtering in only the horizontal dimension, only the vertical dimension, or both. Interpolation operations are implemented using a two-phase polyphase FIR filter.

Decimation operations are implemented using a low-pass two-phase polyphase FIR filter to suppress chroma aliasing.

Features

The Chroma Resampler function converts between different chroma sub-sampling formats. The supported formats are 4:4:4, 4:2:2, and 4:2:0. There are three different options for interpolating and decimating the video samples:

- Define a configurable filter with programmable coefficients for high-performance applications.
- Use the predefined static filter with power-of -two coefficients for low-footprint applications.
- Replicate or drop pixels for minimal footprint.

The following list is a summary of the supported features:

- User programmable filter coefficients
- 4, 6, 8, or 10 tap filter
- Support for RGB, YUV 4:4:4, and YUV 4:2:2, and YUV 4:2:0.
- 8, 10, 12, or 16 bits per component
- Run-time configurable pass through mode if no chroma resampling is needed.

Sub-sampled Video Formats

The sub-sampling scheme is commonly expressed as a three part ratio J:a:b (for example, 4:2:2), that describes the number of luminance and chrominance samples in a conceptual region that is J pixels wide, and 2 pixels high. The parts are (in their respective order):

- J: Horizontal sampling reference (width of the conceptual region). This is usually 4.
- a: Number of chrominance samples (V, U) in the first row of J pixels.
- b: Number of (additional) chrominance samples (V, U) in the second row of J pixels.

To illustrate the most common sub-sampling schemes, [Figure 3-6](#) introduces a graphical notation of sampling grid pixels.

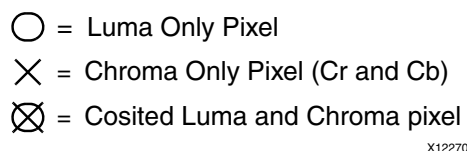


Figure 3-6: Luma, Chroma Notation

4:4:4

Similar to RGB, the 4:4:4 format is used for image capture and display purposes. U and V channels are sampled at the same rate as luminance. Hence, all pixel locations have luma and chroma data co-sited, as shown in Figure 3-7.

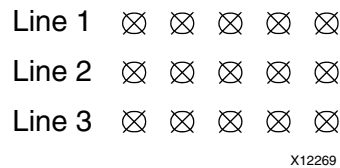


Figure 3-7: YUV 4:4:4 Format

4:2:2

This format contains horizontally sub-sampled chroma. For every two luma samples, there is an associated pair of U and V samples. The sub-sampled chroma locations are co-sited with alternate luma samples as shown in Figure 3-8.

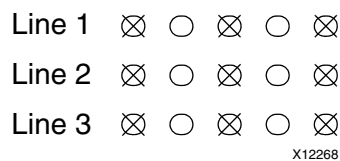


Figure 3-8: YUV 4:2:2 Format

4:2:0

4:2:0 encoding contains horizontally and vertically sub-sampled chroma. Horizontal and vertical chroma positions are co-sited with alternate luma samples on alternate scanlines. The sampling positions are shown in Figure 3-9.



Figure 3-9: YUV 4:2:0 Format

Implementation

Between the three supported sub-sampling formats (4:4:4, 4:2:2, 4:2:0), there are six conversions available. Conversion is achieved using a FIR filter approach. Some require

filtering in only the horizontal dimension or in only the vertical dimension, and in some cases in both the horizontal and the vertical dimensions. These are detailed in Table 3-1 along with default filter information.

Table 3-1: Chroma Resampling Configuration

Converter	Filter Configuration
4:4:4 to 4:2:2	Horizontal anti-aliasing
4:4:4 to 4:2:0	Separable 2-D anti-aliasing
4:2:2 to 4:4:4	Horizontal Interpolation
4:2:2 to 4:2:0	Vertical anti-aliasing
4:2:0 to 4:4:4	Separable 2-D Interpolation
4:2:0 to 4:2:2	Vertical Interpolation

Three implementation options are offered for each conversion operation:

- DSP48 based filter with programmable coefficients and programmable number of taps. 2D filters must be separable. Coefficients are in the range [-8, 8), represented in 16-bit signed, fixed-point format with 4 integer bits and 12 fractional bits.
- The predefined fixed coefficient, non-programmable filter with power of two coefficients (using only shifts and additions for filtering therefore no DSP48s are used). Default coefficients implement linear interpolation for the interpolation and anti-aliasing low pass filters.
- The simplest, lowest footprint solution is to simply drop (decimation) or replicate (interpolation) samples. For down sampling, some samples are passed directly to the output, but others are dropped entirely as appropriate. For up converters, replication of the previous input sample occurs.

Convert 4:2:2 to 4:4:4

This conversion is a 1:2 horizontal interpolation operation, implemented using a two-phase polyphase FIR filter. One of the two output pixels is co-sited with one of the input sample. The ideal output is achieved simply by replicating this input sample.

To evaluate output pixel $o_{x,y}$, the FIR filter convolves $COEFk_HPHASEp_x$, where k is the coefficient index, $i_{x,y}$ are pixels from the input image, p_x is the interpolation phase (0 or 1, depending on x) and $\lfloor \cdot \rfloor_m^M$ represents rounding with clipping at M , and clamping at m . DW is the Data Width or number of bits per video component. N_{taps} is the number of filter taps.

$$o_{x,y} = \left[\sum_{k=0}^{N_{taps}-1} i_{x-k,y} COEFk_HPHASEp_x \right]_{0}^{2^{DW}-1} \quad \text{Equation 3-10}$$

In phase 1, COEF00_HPHASE1 is the coefficient applied to the most recent input sample in the filter aperture. Figure 3-10 illustrates coefficient use for a four tap filter example, with simplified nomenclature a= COEF00_HPHASE1, b= COEF01_HPHASE1, c=COEF02_HPHASE1, and d= COEF03_HPHASE1.

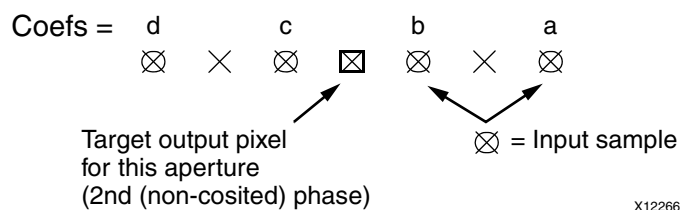


Figure 3-10: 4:2:2 to 4:4:4 Coefficient Configuration

The predefined filters replicate the input sample for Phase 0. The Phase 1 filter is [0.5 0.5].

Convert 4:4:4 to 4:2:2

This conversion is a horizontal 2:1 decimation operation, implemented using a low-pass FIR filter to suppress chroma aliasing. In order to evaluate output pixel $o_{x,y}$, the FIR filter in the core convolves COEFk_HPHASE0, where k is the coefficient index, $i_{x,y}$ are pixels from the input image, and $[]_m^M$ represents rounding with clipping at M, and clamping at m. DW is the Data Width or number of bits per video component. N_{taps} is the number of filter taps.

$$o_{x,y} = \left[\sum_{k=0}^{N_{taps}-1} i_{x-k,y} \text{COEFk_HPHASE0} \right]_0^{2^{DW}-1} \quad \text{Equation 3-11}$$

In phase 0, COEF00_HPHASE0 is the coefficient applied to the most recent input sample in the filter. Figure 3-11 illustrates coefficient use for a 5 tap filter example, with simplified nomenclature a= COEF00_HPHASE0, b= COEF01_HPHASE0, c= COEF02_HPHASE0, d=COEF03_HPHASE0, and e= COEF04_HPHASE0.

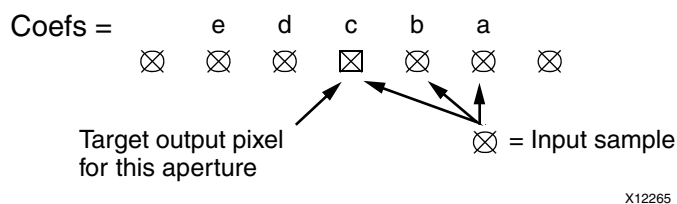


Figure 3-11: 4:4:4 to 4:2:2 Coefficient Configuration

The predefined filter coefficients are [0.25 0.5 0.25].

Convert 4:2:0 to 4:2:2

This conversion is a 1:2 vertical interpolation operation, implemented using a 2-phase polyphase FIR filter. One of the two output pixels is co-sited with one of the input sample. The ideal output is achieved simply by replicating this input sample.

To evaluate output pixel $o_{x,y}$, the FIR filter in the core convolves COEFk_VPASEp_y , where k is the coefficient index, p_y is the interpolation phase, $i_{x,y}$ are pixels from the input image, and $[\cdot]_m^M$ represents rounding with clipping at M , and clamping at m . DW is the Data Width or number of bits per video component. N_{taps} is the number of filter taps.

$$o_{x,y} = \left[\sum_{k=0}^{N_{\text{taps}}-1} i_{x-k,y} \text{COEFk_VPASE0} \right]_0^{2^{DW}-1} \quad \text{Equation 3-12}$$

In phase 1, COEF00_VPASE1 is the coefficient applied to the most recent input sample in the filter. [Figure 3-12](#) illustrates coefficient use for a four tap filter example, with simplified nomenclature $a = \text{COEF00_VPASE1}$, $b = \text{COEF01_VPASE1}$, $c = \text{COEF02_VPASE1}$, and $d = \text{COEF03_VPASE1}$.

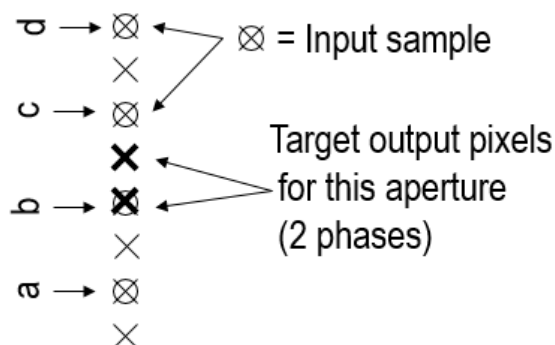


Figure 3-12: 4:2:0 to 4:2:2 Coefficient Configuration

The predefined filters use the coefficients $[0.5 \ 0.5]$ to interpolate one of the output samples. The other output sample is a replication of the input sample.

Convert 4:2:2 to 4:2:0

This conversion is a vertical 2:1 decimation operation, implemented using a low-pass FIR filter to suppress chroma aliasing. In order to evaluate output pixel $o_{x,y}$, the FIR filter in the core convolves COEFk_VPASE0 , where k is the coefficient index, $i_{x,y}$ are pixels from the input image, and $[\cdot]_m^M$ represents rounding with clipping at M , and clamping at m . DW is the Data Width or number of bits per video component. N_{taps} is the number of filter taps.

Equation 3-13

$$o_{x,y} = \left[\sum_{k=0}^{N_{taps}-1} i_{x-k,y} \text{COEFk_VP} \right]_0^{2^{DW}-1}$$

In phase 0, COEF00_VPHASE0 is the coefficient applied to the most recent input sample in the filter. Figure 3-13 illustrates coefficient use for a five tap filter example, with simplified nomenclature a= COEF00_VPHASE0, b= COEF01_VPHASE0, c= COEF02_VPHASE0, d= COEF03_VPHASE0, and e= COEF04_VPHASE0.

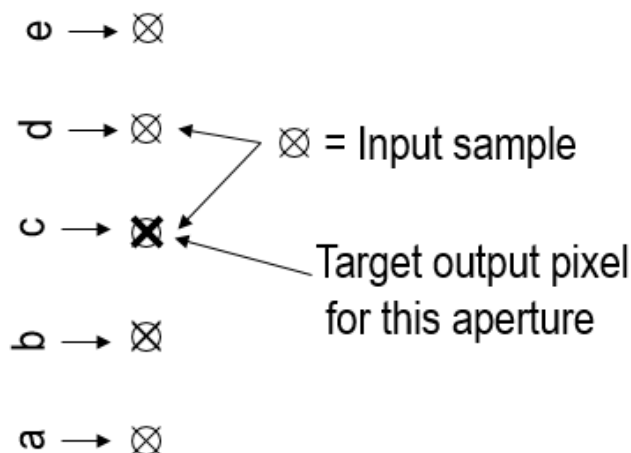


Figure 3-13: 4:2:2 to 4:2:0 Coefficient Configuration

The predefined filter coefficients are [0.25 0.5 0.25].

Convert 4:2:0 to 4:4:4

This conversion performs interpolation both vertically and horizontally. This is equivalent to a 2D separable filter implemented by cascading the 4:2:0 to 4:2:2 block and the 4:2:2 to 4:4:4 block. Quantized vertical filter results are filtered by the horizontal filter, which in turn quantizes results back to the [0 .. 2DW-1] range. (DW is the Data Width or number of bits per video component.)

Intermediate 4:2:2 chroma values are computed using Equation 3-13. The resulting computation is shown in Equation 3-14.

$$t_{x,y} = \left[\sum_{k=0}^{N_{vtaps}-1} i_{x,y-k} \text{COEFk_VP} \right]_0^{2^{DW}-1}$$

Equation 3-14

Next, the values are filtered according to Equation 3-1. The resulting computation is shown in Equation 3-15.

$$o_{x,y} = \left[\sum_{k=0}^{N_{taps}-1} i_{x-k,y} \text{COEFk_HPHASEp}_x \right]_{0}^{2^{DW}-1} \quad \text{Equation 3-15}$$

The predefined filter coefficients are the same as defined in Convert 4:2:0 to 4:2:2 and Convert 4:2:2 to 4:4:4. In the vertical direction, one input sample is replicated, and the other is interpolated with the filter [0.5 0.5]. The same then happens in the horizontal direction.

Convert 4:4:4 to 4:2:0

This conversion performs decimation by 2 both vertically and horizontally. This is equivalent to a 2D separable filter implemented by cascading the 4:4:4 to 4:2:2 block and the 4:2:2 to 4:2:0 block. Quantized horizontal filter results are filtered by the vertical filter, which in turn quantizes results back to the [0 .. 2DW-1] range. (DW is the Data Width or number of bits per video component.)

Intermediate 4:2:2 chroma values are computed using Equation 3-2. The resulting computation is shown in Equation 3-16.

$$t_{x,y} = \left[\sum_{k=0}^{N_{Htaps}-1} i_{x-k,y} \text{COEFk_HPHASE0} \right]_{0}^{2^{DW}-1} \quad \text{Equation 3-16}$$

Next, these values are filtered according to Equation 3-4. The resulting computation is shown in Equation 3-17.

Equation 3-17

$$o_{x,y} = \left[\sum_{k=0}^{N_{Vtaps}-1} t_{x,y-k} \text{COEFk_VPHASE0} \right]_{0}^{2^{DW}-1}$$

The predefined filter coefficients are the same as defined in Convert 4:4:4 to 4:2:2 and Convert 4:2:2 to 4:2:0. In the horizontal direction, decimation is performed with the filter [0.25 0.5 0.25]. The same then happens in the vertical direction.

Resampling Filters

The upsampling and downsampling performed during the chroma format conversion is implemented with low pass filters for the interpolation and anti-aliasing.

The chroma resampling function offers a horizontal filter with a maximum of 10 taps and two phases, as well as a vertical filter with a maximum of 10 taps and two phases. For conversions requiring up/down sampling in both horizontal and vertical directions, 2D separable filters are offered.

The number of taps selected must be even (4, 6, 8, or 10). Depending on the conversion type and filter size selected, a subset of the coefficients can be used by setting the unnecessary coefficients to zero.

Each coefficient has 16 bits in 2's complement format: 4 integer bits (one sign bit) and 12 fractional bits. The sign bit is the MSB. For example, a coefficient with a value of 1 is represented with this bit vector

```
[0001000000000000].
```

The coefficients should sum to exactly 1 to achieve unity gain. If they sum to less than 1, some loss of dynamic range is observed.

Computation Bit Width Growth

Full precision ($\text{DATA_WIDTH} + 16 + \log_2(N_{\text{Taps}})$ bits) is maintained during the horizontal and/or vertical FIR convolution operation.

FIR filter outputs are rounded to DATA_WIDTH bits by adding half an output LSB in the full precision domain prior to truncation. Clipping and clamping of the output data prevents overflows and underflows. Data is clipped and clamped at $2^{\text{DATA_WIDTH}-1}$ and 0.

Edge Padding

The edge pixels of images are replicated prior to filtering to avoid image artifacts.

Clocking

The AXI Streaming, AXI Memory-Mapped, and AXI Lite interfaces can be run at their own clock rate. Therefore, three separate clock interfaces are provided named `aclk_axis`, `aclk_axi_mm`, and `aclk_ctrl`, respectively.

Pixel throughput of the Video Processing Subsystem is defined by the product of the clock frequency times the **Samples per Clock** setting in the GUI. With a clock frequency of 300 MHz for `aclk_axis`, and a two sample per clock configuration, the Video Processing Subsystem is capable of a 600 mega pixel throughput rate, which is sufficient to handle 4K resolutions at 60 Hz.

Resets

The `aresetn_ctrl` signal is the active-Low reset input signal of the IP. The reset signal must be synchronous to the `aclk_ctrl` signal. Each time this reset is asserted, it should be asserted for a minimum of 16 clock cycles of the slowest clock. All registers are reset to power-on conditions; all queues are flushed; all internal logic returned to power-on conditions.

The `aresetn_io_axis` signal is an outgoing signal that can be used to hold IPs in reset when the Video Processing Subsystem is not ready to consume data on the streaming input. This reset signal is synchronous to the `aclk_axis` signal.

Design Flow Steps

This chapter describes customizing and generating the core, constraining the core, and the simulation, synthesis and implementation steps that are specific to this IP core. More detailed information about the standard Vivado® design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994) [\[Ref 1\]](#)
- *Vivado Design Suite User Guide: Designing with IP* (UG896) [\[Ref 2\]](#)
- *Vivado Design Suite User Guide: Getting Started* (UG910) [\[Ref 3\]](#)
- *Vivado Design Suite User Guide: Logic Simulation* (UG900) [\[Ref 4\]](#)

Customizing and Generating the Core

This section includes information about using Xilinx tools to customize and generate the core in the Vivado Design Suite.

The Video Processing Subsystem can only be added to a Vivado IP integrator block design in the Vivado Design Suite. For more detailed information on customizing and generating the core in the Vivado IP integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* (UG994) [\[Ref 1\]](#). IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

1. In the Flow Navigator, click on **Create Block Diagram** or **Open Block Design** under the IP Integrator heading.
2. Right click in the diagram and select **Add IP**.

A searchable IP catalog opens. You can also add IP by clicking on the Add IP button on the left side of the IP Integrator Block Design canvas.

3. Click on the IP name and press the **Enter** key on your keyboard or double click on the IP name.
4. Double-click the selected IP block or select the **Customize Block** command from the right-click menu.

For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896) [Ref 2] and the *Vivado Design Suite User Guide: Getting Started* (UG910) [Ref 3].

Note: Figures in this chapter are illustrations of the Vivado Integrated Design Environment (IDE). The layout depicted here might vary from the current version.

Top Level Settings Tab

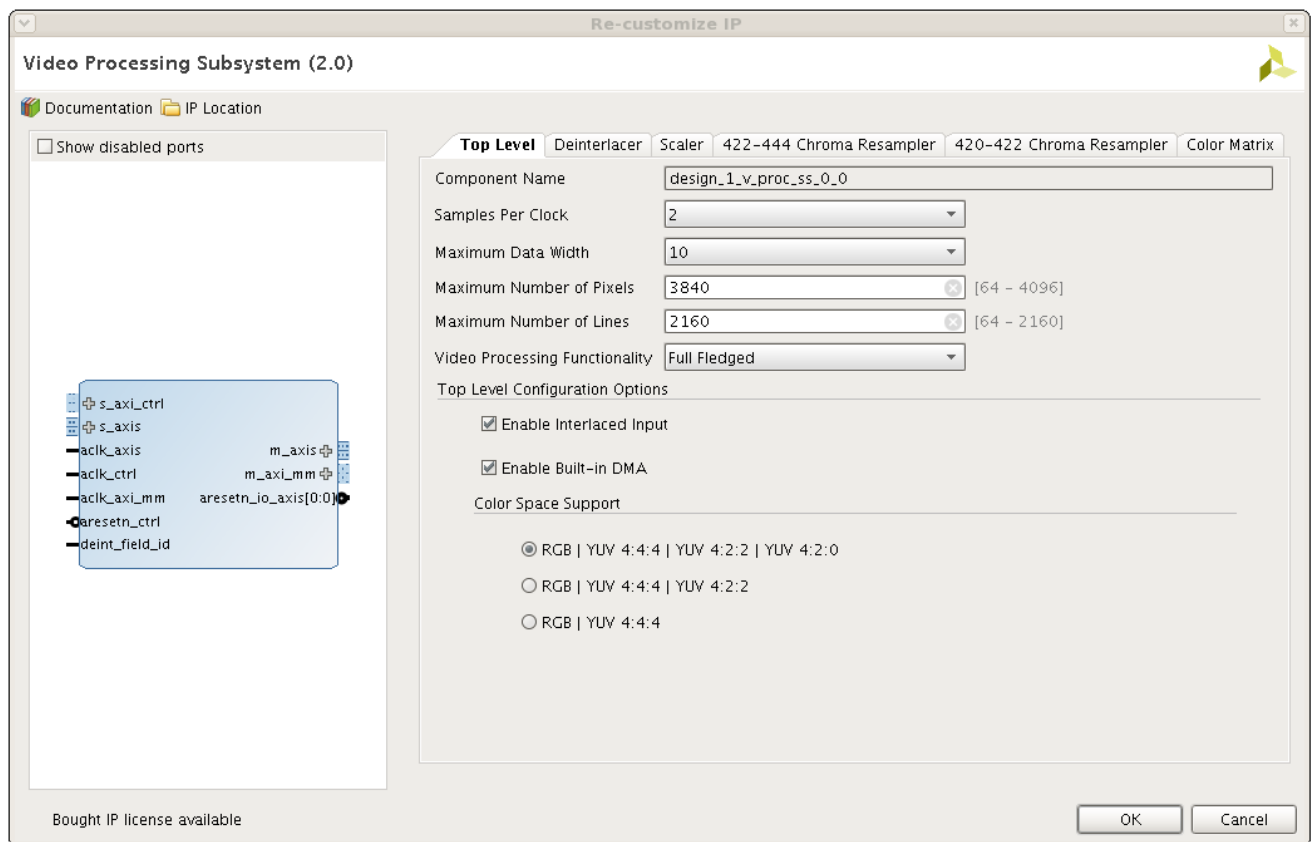


Figure 4-1: Top Level Tab

The parameters on the Top-level tab are as follows:

- **Component Name:** Component name is set automatically by IP Integrator.
- **Samples Per Clock:** Select 1, 2, or 4 pixel wide interface.
- **Maximum Data Width:** Select 8, 10, 12, or 16 bit color depth.
- **Maximum Number of Pixels:** Specify the maximum number of pixels per scanline. Permitted values are from 64 to 4096. Specifying this value is necessary to establish the

depth of internal line buffers. Using a tight upper-bound results in optimal block RAM usage. The active frame size can be programmed during run-time via the AXI4-Lite interface and driver API. The number of active columns must be less than or equal to the Maximum Number of Pixels.

- **Maximum Number of Lines:** Specify the maximum number of scanlines per frame. Permitted values are from 64 to 2160. The active frame size can be programmed during run-time via the AXI4-Lite interface and driver API. The number of active rows must be less than or equal to the Maximum Number of Lines.
- **Video Processing Functionality:** The following sets of processing functionality are offered:
 - **Full Fledged:** The Full Fledged design indicates deinterlacing (optional), scaling, color matrix operations such as color space conversion and correction, chroma resampling (optional), and frame rate conversion by means of drop/repeat (optional).
 The Full Fledged configuration is the most resource intensive configuration, but allows for stripping down the full configuration by excluding deinterlacing, frame rate conversion, and chroma resampling. If deinterlacing is not included, only progressive inputs can be supported. Similarly, the Color Space Support option (explained below), allows for stripping down the Video Processing Subsystem to handle only RGB and YUV 4:4:4, therefore eliminating the need for horizontal and vertical chroma resampling.
 In the Full Fledged configuration, you have full configurability over the included functions, e.g. the algorithm and/or number of taps used for chroma resampling can be changed.
 - **Scaler Only:** The subsystem is configured to perform the scaling function with an option for video format conversion between RGB, YUV 4:4:4, YUV 4:2:2, and YUV 4:2:0.
 Note that together with Enable Color Space Conversion, and Color Space Support for RGB, YUV 4:4:4, YUV 4:2:2, YUV 4:2:0, the Scaler Only configuration is effectively a scaler with any to any format conversion. In terms of resources, this is a cost-effective alternative to the Full Fledged configuration as long as there is no need for better than basic chroma resampling quality and no need for programming the color space conversion coefficients.
 - **Deinterlacing Only:** The subsystem is configured to perform only the deinterlacing function.
 - **Color Space Conversion Only:** The subsystem is configured to perform only color space conversion and correction functions.
 - **420-422 Chroma Resampling Only:** The subsystem is configured to perform only the vertical chroma resampling function.
 - **422-444 Chroma Resampling Only:** The subsystem is configured to perform only the horizontal chroma resampling function.

- **Top Level Configuration Options:** When the Full Fledged Video Processing Functionality is selected, the following additional configurations are available:
 - **Enable Interlaced Input:** Select this checkbox if interlaced support is desired. In this case, the subsystem includes the Deinterlacer subcore.
 - **Enable Built-in DMA:** Select this checkbox to include a Video DMA Engine in the subsystem.
 - **Color Space Support:** Select the color spaces for which support is desired. If YUV 4:2:2 is selected, the Horizontal Chroma Resampler subcore is included. If YUV 4:2:0 is selected, the Vertical Chroma Resampler subcore is also included.

Deinterlacer Settings

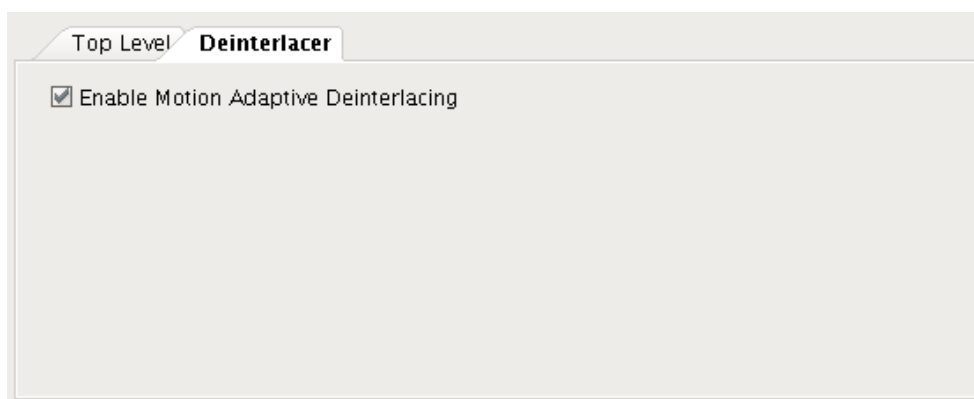
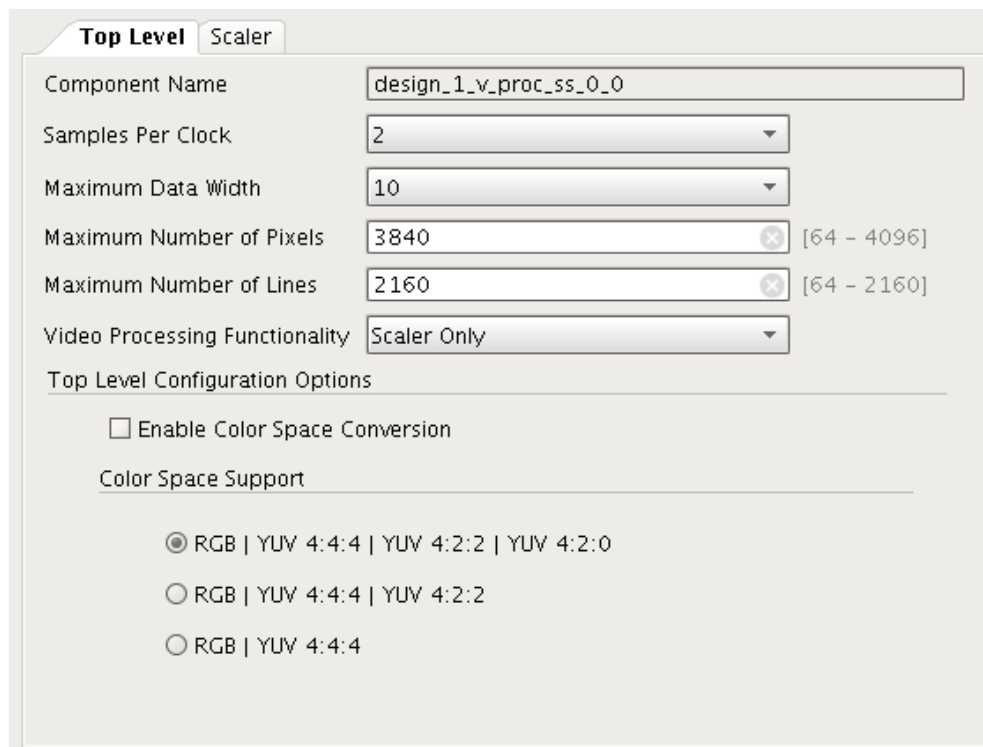


Figure 4-2: Deinterlacer Tab

The parameter on the Deinterlacer tab is as follows:

- **Enable Motion Adaptive Deinterlacing:** Selecting this option allows support for all six deinterlacing algorithms: Line Doubling, Weave, Vertical Temporal Linear Interpolation, Vertical Temporal Median, Median, Bilinear Interpolation. If this checkbox is not selected, only the following two algorithms are supported: Bob and Bilinear Interpolation.

Scaler Settings



Top Level | Scaler

Component Name: design_1_v_proc_ss_0_0

Samples Per Clock: 2

Maximum Data Width: 10

Maximum Number of Pixels: 3840 [64 – 4096]

Maximum Number of Lines: 2160 [64 – 2160]

Video Processing Functionality: Scaler Only

Top Level Configuration Options

☐ Enable Color Space Conversion

Color Space Support

☒ RGB | YUV 4:4:4 | YUV 4:2:2 | YUV 4:2:0

☐ RGB | YUV 4:4:4 | YUV 4:2:2

☐ RGB | YUV 4:4:4

Figure 4-3: Top Level Scaler Options

When the Video Processing Functionality is chosen to be Scaler Only, the following options are available:

- **Enable Color Space Conversion:** When selected, the Scaler Only subcore supports run-time configurable color space conversion from RGB to YUV 4:4:4 and vice versa between the input and output video streams. Note that the coefficients are fixed and not programmable.
- **Color Space Support:** Select the color spaces for which support is desired. If YUV 4:2:2 is selected, basic horizontal chroma resampling is included. If YUV 4:2:0 is selected, basic vertical chroma resampling is also included. The combination of Enable Color Space Conversion and Color Space Support for RGB, YUV 4:4:4, YUV 4:2:2, YUV 4:2:0 basically turns the Scaler Only configuration into a scaler with any to any format conversion.

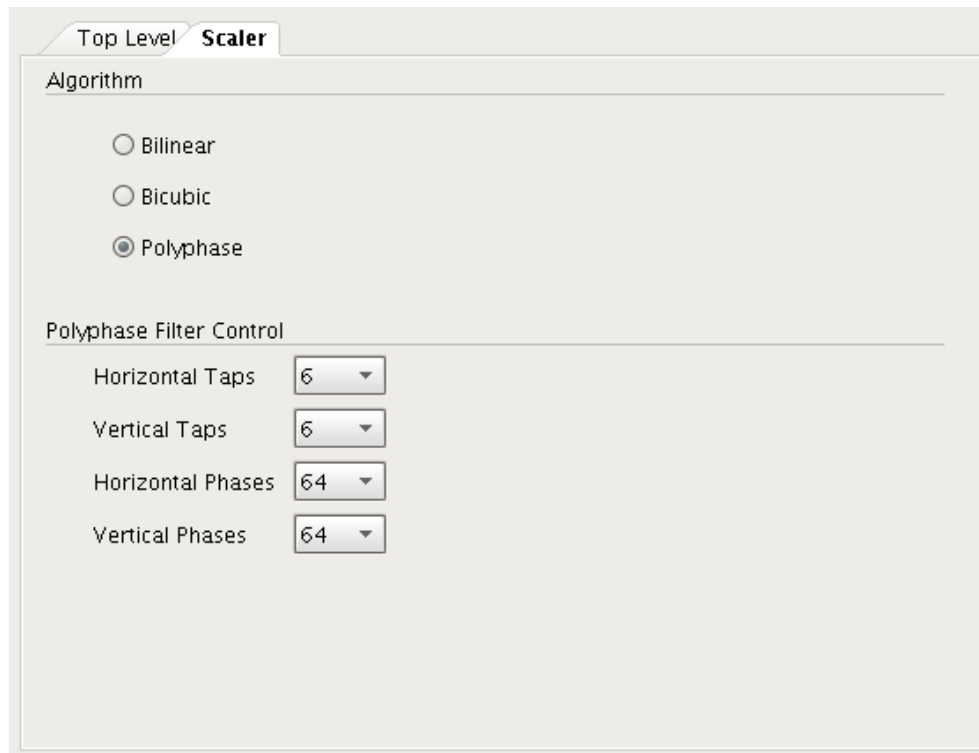


Figure 4-4: Scaler Tab

The parameters on the Scaler tab are as follows:

- **Algorithm:** The Scaler comes in three different quality levels each at different levels of resource usage.
 - **Bilinear:** Bilinear scaling uses bilinear interpolation to calculate pixels.
 - **Bicubic:** Bicubic scaling is a little bit more demanding compared to bilinear scaling, and produces smoother pictures with less artifacts.
 - **Polyphase:** The picture quality (and resource usage) of a polyphase scaler depends largely on the number of filter taps used and number of filter phases used.
- **Polyphase Filter Control:** When polyphase filtering is selected, the horizontal and vertical taps and phases are defined here.
 - **Horizontal Taps:** Select 6, 8, 10, or 12 horizontal filter taps.
 - **Vertical Taps:** Select 6, 8, 10, or 12 vertical filter taps.
 - **Horizontal Phases:** Only 64 phases is currently supported.
 - **Vertical Phases:** Only 64 phases is currently supported.

Chroma Resampler Settings

The Video Processing Subsystem supports chroma resampling between 4:4:4 and 4:2:2 formats as well as between 4:2:2 and 4:2:0.

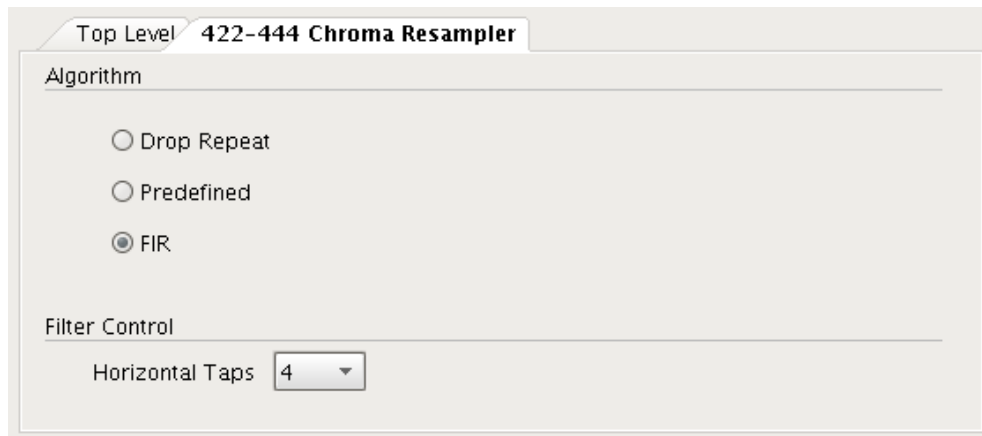


Figure 4-5: 4:2:2 – 4:4:4 Chroma Resampling Tab

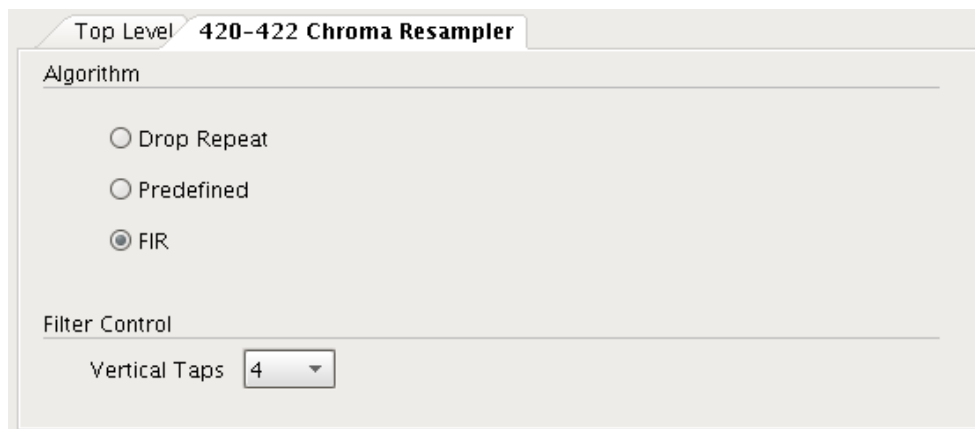


Figure 4-6: 4:2:0 – 4:2:2 Chroma Resampling Tab

The parameters on the Chroma Resampler tabs are as follows:

- **Algorithm:**
 - **Drop Repeat:** Using the drop option results in down conversion with no filter. Some samples are passed directly to the output, but others are dropped entirely, as appropriate. This occurs on a line-by-line basis and on a pixel-by-pixel basis. The replicate option is available in all upconverters. It applies in both vertical and horizontal domains as appropriate. Using the replicate option results in up conversion with no filter. Replication of the previous input sample occurs instead.
 - **Predefined:** Filters are predefined and not programmable. The filters use only power-of-two coefficients so no DSP48s are necessary. Linear interpolation is employed for the low pass filters used for anti-aliasing and interpolation.

- **FIR:** Users can program the filter coefficients through the AXI4-Lite interface.
- **Filter Control:** When FIR filtering is selected, the number of Horizontal and Vertical taps are defined here.
 - **Horizontal Taps:** Select 4, 6, 8, or 10 horizontal filter taps.
 - **Vertical Taps:** Select 4, 6, 8, or 10 vertical filter taps.

Color Matrix Settings

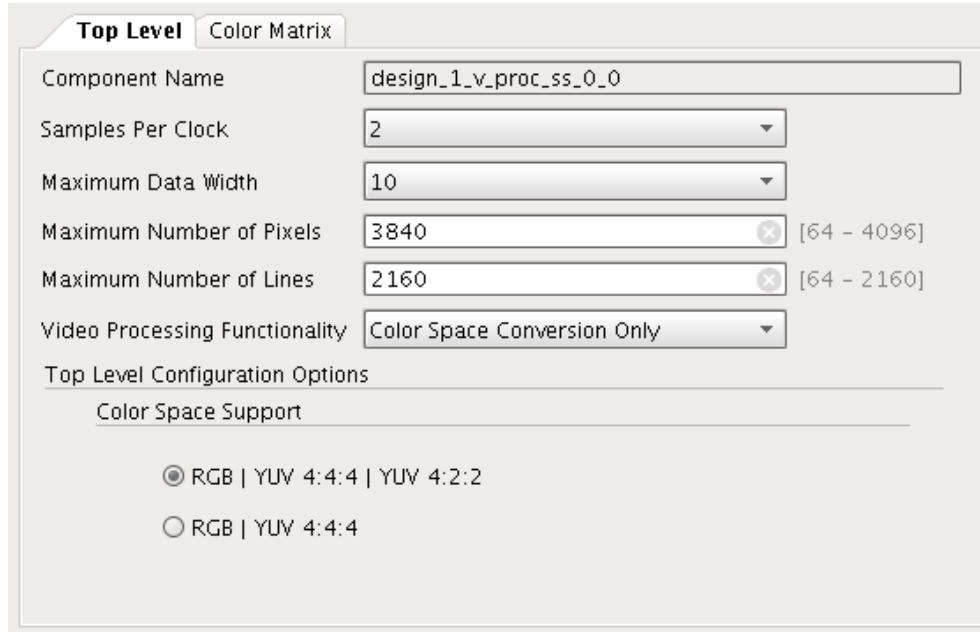


Figure 4-7: Top Level Color Matrix Option

When **Color Space Conversion Only** is selected in **Video Processing Functionality**, the following option is available:

- **Color Space Support:** Select the color spaces for which support is desired. If YUV 4:2:2 is selected, basic horizontal chroma resampling is included. For YUV 4:2:2, both input and output video streams will be in YUV 4:2:2 format.



Figure 4-8: Color Matrix Tab

The parameter on the Color Matrix tab is:

- **Enable Demo Window:** This feature allows a different set of matrix coefficients to be used within a user-defined window in the image.

User Parameters

Table 4-1 shows the relationship between the fields in the Vivado IDE and the User Parameters (which can be viewed in the Tcl Console).

Table 4-1: Vivado IDE Parameter to User Parameter Relationship

Vivado IDE Parameter/Value ⁽¹⁾	User Parameter/Value ⁽¹⁾	Default Value
Top Level		
Samples Per Clock	C_SAMPLES_PER_CLK	2
Maximum Data Width	C_MAX_DATA_WIDTH	10
Maximum Number of Pixels	C_MAX_COLS	3840
Maximum Number of Lines	C_MAX_ROWS	2160
Video Processing Functionality	C_TOPOLOGY	1
Scaler Only	0	
Full Fledged	1	
Deinterlacing Only	2	
Color Space Conversion Only	3	
420-422 Chroma Resampling Only	4	
422-444 Chroma Resampling Only	5	

Table 4-1: Vivado IDE Parameter to User Parameter Relationship (Cont'd)

Vivado IDE Parameter/Value ⁽¹⁾	User Parameter/Value ⁽¹⁾	Default Value
Enabled Interlaced Input	C_ENABLE_INTERLACED	true
Enable Build-in DMA	C_ENABLE_DMA	true
Color Space Support	C_COLORSPACE_SUPPORT	0
RGB YUV 4:4:4 YUV 4:2:2 YUV 4:2:0	0	
RGB YUV 4:4:4 YUV 4:2:2	1	
RGB YUV 4:4:4	2	
Deinterlacer		
Enable Motion Adaptive Deinterlacing	C_DEINT_MOTION_ADAPTIVE	true
Scaler		
Algorithm	C_SCALER_ALGORITHM	Polyphase
Bilinear	0	
Bicubic	1	
Polyphase	2	
Horizontal Taps	C_H_SCALER_TAPS	6
Vertical Taps	C_V_SCALER_TAPS	6
Horizontal Phases	C_H_SCALER_PHASES	64
Vertical Phases	C_V_SCALER_PHASES	64
Enable 4:2:2 Color Format	C_SCALER_ENABLE_422	true
Chroma Resampler		
Horizontal Algorithm	C_H_CHROMA_ALGORITHM	FIR
Drop Repeat	0	
Predefined	1	
FIR	2	
Vertical Algorithm	C_V_CHROMA_ALGORITHM	FIR
Drop Repeat	0	
Predefined	1	
FIR	2	
Horizontal Taps	C_H_CHROMA_TAPS	4
Vertical Taps	C_V_CHROMA_TAPS	4
Color Matrix		
Enable Demo Window	C_CSC_ENABLE_WINDOW	true
Enable 4:2:2 Color Format	C_CSC_ENABLE_422	true

Notes:

- Parameter values are listed in the table where the Vivado IDE parameter value differs from the user parameter value. Such values are shown in this table as indented below the associated parameter.

Output Generation

For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896) [Ref 2].

Constraining the Core

This section contains information about constraining the core in the Vivado Design Suite.

Required Constraints

The only constraints required are clock frequency constraints for the AXI4-Stream video interfaces clock, `ac1k_axis`, AXI4-Lite control interface clock, `ac1k_ctrl`, and memory subsystem clock, `ac1k_axi_mm`. Paths from AXI4-Lite signals should be constrained with a `set_false_path`, causing setup and hold checks to be ignored for AXI4-Lite signals. These constraints are provided in the XDC constraints files included with the IP.

Device, Package, and Speed Grade Selections

This section is not applicable for this IP core.

Clock Frequencies

This section is not applicable for this IP core.

Clock Management

This section is not applicable for this IP core.

Clock Placement

This section is not applicable for this IP core.

Banking

This section is not applicable for this IP core.

Transceiver Placement

This section is not applicable for this IP core.

I/O Standard and Placement

This section is not applicable for this IP core.

Simulation

Simulation of this core is not supported.

Synthesis and Implementation

For details about synthesis and implementation, see the *Vivado Design Suite User Guide: Designing with IP* (UG896) [\[Ref 2\]](#).

Detailed Example Design

An IP Integrator example design is provided to demonstrate the Video Processing Subsystem capabilities.



IMPORTANT: *The example project is only available on Xilinx KC705 evaluation board.*

To open the example project, perform the following:

1. Add the IP to an IP Integrator Block Design canvas.
2. After the IP has been instantiated in the block design, double-click on it to customize it.
3. Right-click the selected IP block and select **Open IP Example Design** from the menu.
4. In the Open IP Example Design window, select example project directory and click **OK**.

A new session of the Vivado® IDE opens that shows the example design in the Design Sources window.

Full Fledged and Scaler Only Video Processing Design

Figure 5-1 shows the top level block diagram. In the video path is a video test pattern generator and an AXI4-Stream to Video Out core. Furthermore, a MicroBlaze™ processor is controlling the IPs, and a memory interconnect with MIG are interfacing with external DDR.

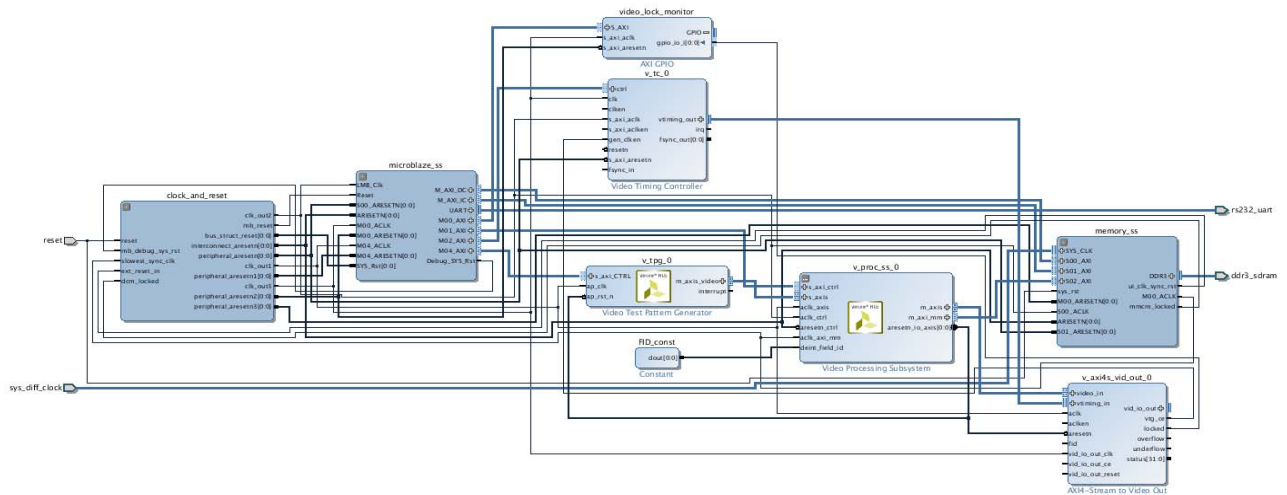


Figure 5-1: Top-level Full Fledged or Scaler Only Video Processing Design

Deinterlacing, Color Space Conversion, or Chroma Resampling Only Video Processing Design

Figure 5-2 shows the top level block diagram of the Deinterlacing, Color Space Conversion, or Chroma Resampling only design. The main difference between this design and the Full Fledged/Scaler Only design is that there is a GPIO included as an external reset for the Video Processing Subsystem. The Full Fledged and Scaler Only configurations contain a GPIO as a reset as part of the subsystem.



A video test pattern generator feeds into the Video Processing Subsystem. The output of the Video Processing Subsystem is connected to the AXI4-Stream to Video Out IP.

The memory subsystem in the Full Fledged and Deinterlacing Only designs consists of an AXI-MM interconnect that is a 3:1 cross-bar that feeds into the MIG. The 3 ports feeding into the cross bar are the data and instruction cache ports from the Microblaze, and the memory port of the Video Processing Subsystem. For the Scaler Only, Color Space Conversion Only, and Chroma Resampling Only designs, the interconnect is a 2:1 cross bar.

A MicroBlaze processor is used for controlling the IPs. The example design uses the default configuration which is not optimized for performance as CPU load is not critical. The MicroBlaze has 8KB data and instruction caches added. The MicroBlaze processor is being run at 100 MHz clock speed.

The Video Processing Subsystem uses three clocks that are derived through a clock generator from the 200 MHz system clock that is available on the Kintex®-7 KC705 board. The memory subsystem is running at this system clock at 200 MHz. This system clock feeds into the clock wizard IP, which derives a 300 MHz and 100 MHz clock at the output. The 300

MHz clock drives the AXI4-Stream video interfaces. The 100 MHz clock drives the AXI4-Lite control interface, and also drives the MicroBlaze processor.

Example Design Software

The synthesizable example design requires both Vivado and Xilinx® SDK tools.

The first step is to run synthesis, implementation and bitstream generation in Vivado. After all those steps are done, select **File -> Export -> Export Hardware**. In the window, select **Include bitstream**, select an export directory and click **OK**.

The remaining work is performed in Xilinx SDK tool. The example design file can be found at SDK directory:

```
(<install_directory>/SDK/<Vivado version>/data/embeddedsw/XilinxProcessorIPLib/  
drivers/vprocss_v2_1/examples/
```

Example application design source files (contained within "examples" folder) are tightly coupled with the example design available in Vivado Catalog.

vpss_example.tcl automates the process of generating the downloadable bit and elf files from the provided example hdf file.

To run the provided Tcl script:

1. Copy the exported example design hdf file in the "examples" directory of the driver
2. Launch the Xilinx Software Command-Line Tool (xsct) terminal
3. cd into the examples directory
4. Source the tcl file

```
xsct%>source vpss_example.tcl
```

5. Execute the script

```
xsct%>vpss_example <hdf_file_name.hdf>
```

The Tcl script performs the following:

- Create workspace
- Create HW project
- Create BSP
- Create Application Project
- Build BSP and Application Project

After the process is complete, the required files are available in:

```
bit file -> vpss_example.sdk/vpss_example_hw_platform folder
```

```
elf file -> vpss_example.sdk/vpss_example_design/{Debug/Release} folder
```

Next, perform the following steps to run the software application:



IMPORTANT: *To do so, make sure that the hardware is powered on and a Digilent Cable or an USB Platform Cable is connected to the host PC. Also, ensure that a USB cable is connected to the UART port of the KC705 board.*

1. Launch SDK.
 2. Set workspace to vpss_example.sdk folder in prompted window. The SDK project opens automatically. (If a welcome page shows up, close that page.)
 3. Download the bitstream into the FPGA by selecting **Xilinx Tools > Program FPGA**. The Program FPGA dialog box opens.
 4. Ensure that the Bitstream field shows the bitstream file generated by Tcl script, and then click **Program**.
- Note:** The DONE LED on the board turns green if the programming is successful.
5. A terminal program (HyperTerminal or PuTTY) is needed for UART communication. Open the program, choose appropriate port, set baud rate to 9600 and establish Serial port connection.
 6. Select and right-click the application vpss_example_design in Project_Explorer panel.
 7. Select **Run As > Launch on Hardware (GDB)**.
 8. Select Binaries and Qualifier in window and click **OK**.

The example design test result are shown in terminal program.

For more information, visit www.xilinx.com/tools/sdk.htm.

When executed on the board, the example application determines the Video Processing Subsystem topology and sets the input and output stream configuration accordingly. The test pattern generator IP is used to generate the input stream. Video Lock Monitor IP then monitors the output of the subsystem (to vidout) to determine if lock is achieved and present the status (Pass/Fail) on the terminal.

Migrating and Upgrading

This appendix contains information about upgrading to a more recent version of the IP core.

Upgrading in the Vivado Design Suite

Parameter Changes

The Video Processing Subsystem v2.0 supports more configurability for the underlying subcores. Version 2.0 supports Deinterlacing Only, Color Space Conversion Only, and Chroma Resampling Only configurations.

The default Video Processing functionality has changed from Scaler Only to Full Fledged.

Chroma Resampling is now divided into 4:4:4-4:2:2 and 4:2:2-4:2:0. The algorithm and filter taps can be selected independently in the horizontal and vertical directions.

The drivers have been updated for v2.0. For more information, see [Appendix C, Application Software Development](#).

Debugging

This appendix includes details about resources available on the Xilinx Support website and debugging tools.



TIP: *If the IP generation halts with an error, there might be a license issue. See [License Checkers in Chapter 1](#) for more details.*

Finding Help on Xilinx.com

To help in the design and debug process when using the Video Processing Subsystem, the [Xilinx Support web page](#) contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support.

Documentation

This product guide is the main document associated with the Video Processing Subsystem. This guide, along with documentation related to all products that aid in the design process, can be found on the [Xilinx Support web page](#) or by using the Xilinx Documentation Navigator.

Download the Xilinx Documentation Navigator from the [Downloads page](#). For more information about this tool and the features available, open the online help after installation.

Solution Centers

See the [Xilinx Solution Centers](#) for support on devices, software tools, and intellectual property at all stages of the design cycle. Topics include design assistance, advisories, and troubleshooting tips.

The Solution Center specific to the Video Processing Subsystem core is listed below.

- [Xilinx Multimedia, Video and Imaging Solution Center](#)

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with a Xilinx product. Answer Records are created and maintained daily ensuring that users have access to the most accurate information available.

Answer Records for this core can be located by using the Search Support box on the main [Xilinx support web page](#). To maximize your search results, use proper keywords such as

- Product name
- Tool message(s)
- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Master Answer Record for the Video Processing Subsystem

AR: [65449](#)

Technical Support

Xilinx provides technical support at the [Xilinx Support web page](#) for this LogiCORE™ IP product when used as described in the product documentation. Xilinx cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To contact Xilinx Technical Support, navigate to the [Xilinx Support web page](#).

Debug Tools

There are many tools available to address Video Processing Subsystem design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The Vivado® Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in Xilinx devices.

The Vivado logic analyzer is used with the logic debug IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* (UG908) [Ref 6].

Reference Boards

Various Xilinx development boards support the Video Processing Subsystem. These boards can be used to prototype designs and establish that the core can communicate with the system. The Video Processing Subsystem offers an Example Design which Xilinx recommends for getting acquainted with the core.

- 7 series FPGA evaluation boards
 - KC705

Simulation Debug

Simulation of this core is not supported.

Hardware Debug

Hardware issues can range from link bring-up to problems seen after hours of testing. This section provides debug steps for common issues. Vivado Lab Edition is a valuable resource to use in hardware debug. The signal names mentioned in the following individual sections can be probed using Vivado Lab Edition for debugging the specific problems.

General Checks

Ensure that all the timing constraints for the core were properly incorporated from the example design and that all constraints were met during implementation.

- Does it work in post-place and route timing simulation? If problems are seen in hardware but not in timing simulation, this could indicate a PCB issue. Ensure that all clock sources are active and clean.
- If using MMCMs in the design, ensure that all MMCMs have obtained lock by monitoring the `locked` port.
- If your outputs go to 0, check your licensing.

Interface Debug

AXI4-Lite Interfaces

Read from a register that does not have all 0s as a default to verify that the interface is functional. Output `s_axi_arready` asserts when the read address is valid, and output `s_axi_rvalid` asserts when the read data/response is valid. If the interface is unresponsive, ensure that the following conditions are met:

- The `s_axi_aclk` and `aclk` inputs are connected and toggling.
- The interface is not being held in reset, and `s_axi_areset` is an active-Low reset.
- The interface is enabled, and `s_axi_aclken` is active-High (if used).
- The main core clocks are toggling and that the enables are also asserted.
- If the simulation has been run, verify in simulation and/or a Vivado Lab Edition capture that the waveform is correct for accessing the AXI4-Lite interface.

AXI4-Stream Interfaces

If data is not being transmitted or received, check the following conditions:

- If transmit `<interface_name>_tready` is stuck Low following the `<interface_name>_tvalid` input being asserted, the core cannot send data.
- If the receive `<interface_name>_tvalid` is stuck Low, the core is not receiving data.
- Check that the `aclk` inputs are connected and toggling.
- Check that the AXI4-Stream waveforms are being followed.
- Check core configuration.

Application Software Development

Driver

The Video Processing Subsystem driver abstracts the included video processing elements and presents the processing chain as a black-box. It shields you from the internal workings of included subcores and thus provides an out-of-the-box solution for video

The subsystem driver is a bare-metal driver. It dynamically manages the data and control flow through the processing elements, based on the input/output stream configuration set at run time. Internally, it relies on included subcore layer 1 drivers to configure the IP hardware block and layer 2 drivers to provide an abstracted view of the feature set provided by each subcore.

Dependencies

A video common driver is delivered as part of the SDK. Most video IPs have master/slave AXI4-Stream interfaces so the concept of Video Streams as an interface parameter for drivers for such cores was introduced.

The video_common defines the following features:

- Enumerations for video specific details like color format, color depth, frame rate, etc.
- Video stream, video timing and video window data types.
- Video Mode Table with predefined resolutions and their timing details.
- Utility APIs that can access data from the mode table and work with stream data types.

Architecture

The subsystem driver provides an easy-to-use, well-defined API to help integrate the subsystem in an application without having to understand the underlying complexity of configuring each and every subcore.

Subsystem driver consists of the following:

- Subsystem layer: Controls the data/control flow of AXI4-Stream through the processing cores. It uses support functions to examine the input and output stream properties and make appropriate decision to implement the determined use case.
- Subcore drivers: Every included subcore has a driver associated with it and provides 2 layers of abstraction
 - Layer-1: Implements API's to peek/poke IP registers at HW level.
 - Layer-2: Implements feature set to abstract core functionality.

For example, the color space conversion IP at the core level contains a 3 x 3 coefficient matrix which must be manipulated to do the required conversion, for example RGB to YUV.

- Layer-1 provides APIs to read/write these 3 x 3 coefficient registers. You are responsible for computing the required coefficient values and program the specific registers.
- Layer-2 implements the feature set that can be built upon layer-1 and is much easier to use:
 - API to convert RGB to YUV or vice-versa.
 - API to set/get brightness, contrast, saturation and gain.
 - Auto translation between user setting of 0-100 to IP offered range.

Figure C-1 shows the Video Processing Subsystem architecture.

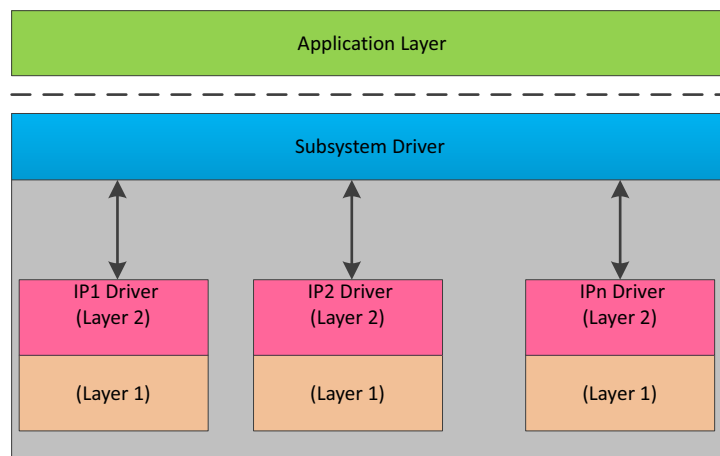


Figure C-1: Subsystem Driver Architecture

Prerequisites

For Full Fledged topology, the subsystem requires external memory to store video buffers. You must specify the DDR address for the buffer storage. The API to use is:

```
void XVprocSs_SetFrameBufBaseaddr(XVprocSs *InstancePtr, UINTPTR addr);
```

Usage

The subsystem driver itself is not an active driver and relies on application software to make use of the provided APIs to configure it. Application software is responsible to monitor the system for external inputs and communicate changes to input/output stream properties to the subsystem through provided APIs, and trigger the subsystem auto reconfiguration process.

To integrate and use the Video Processing Subsystem driver in a user application, the following steps must be followed:

1. Include the subsystem header file xvprocss.h that defines the subsystem object.

2. Declare an instance of the subsystem object in the application code:

```
XVprocSs VprocInst;
```

3. Initialize system peripherals (timer, interrupt controller, UART, etc.) in the application.
4. If delay routine is defined at application level using a timer, this must be registered with the subsystem:

```
void XVprocSs_SetUserTimerHandler(XVprocSs *InstancePtr,
                                   XVidC_DelayHandler CallbackFunc,
                                   void *CallbackRef);
```

If the application is not using the timing peripheral to implement the timeouts, skip this step. The Subsystem driver internally uses the platform-specific delay handler.

5. Initialize Video Processing Subsystem at Power-On.

```
int XVprocSs_CfgInitialize(XVprocSs *InstancePtr,
                           XVprocSs_Config *CfgPtr,
                           UINTPTR EffectiveAddr);
```

Accesses the HW configuration determines the included subcores and initializes them to the power on default. The input and output stream resolution is set to 1080p at 60 Hz, RGB, and color depth is 10-bit. Finally all the cores are reset and the subsystem Ready flag is set.

6. Configure subsystem:

```
int XVprocSs_SetSubsystemConfig(XVprocSs *InstancePtr);
```

This API is the entry point and trigger mechanism for the subsystem processing control flow. It examines the input and output stream properties and validates them against the supported feature set. If the use case is supported, the driver determines the subcores required to implement the use case, builds a data flow network, configures cores in the network, and starts the video pipe.

Now, the subsystem is ready for use and the application software monitors changes in the system input or output and communicates those changes to the video subsystem.

Additional Resources and Legal Notices

Xilinx Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see [Xilinx Support](#).

References

These documents provide supplemental material useful with this product guide:

1. *Vivado® Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
2. *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
3. *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
4. *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))
5. *ISE to Vivado Design Suite Migration Guide* ([UG911](#))
6. *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#))
7. *Vivado Design Suite User Guide: Implementation* ([UG904](#))
8. *AXI Reference Guide* ([UG1037](#))
9. *AXI Video Direct Memory Access Product Guide* ([PG020](#))

Revision History

The following table shows the revision history for this document.

Date	Version	Revision
10/05/2016	2.0	Added video format conversion to Scaler Only configuration, added YUV 4:2:0 support to Video Deinterlacer, added pass through mode to Chroma Resamplers. Updated Xilinx automotive applications disclaimer.
04/06/2016	2.0	Additional configurability features.
11/18/2015	1.0	Added support for UltraScale+ devices.
10/19/2015	1.0	Initial Xilinx release.

Please Read: Important Legal Notices

The information disclosed to you hereunder (the "Materials") is provided solely for the selection and use of Xilinx products. To the maximum extent permitted by applicable law: (1) Materials are made available "AS IS" and with all faults, Xilinx hereby DISCLAIMS ALL WARRANTIES AND CONDITIONS, EXPRESS, IMPLIED, OR STATUTORY, INCLUDING BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE; and (2) Xilinx shall not be liable (whether in contract or tort, including negligence, or under any other theory of liability) for any loss or damage of any kind or nature related to, arising under, or in connection with, the Materials (including your use of the Materials), including for any direct, indirect, special, incidental, or consequential loss or damage (including loss of data, profits, goodwill, or any type of loss or damage suffered as a result of any action brought by a third party) even if such damage or loss was reasonably foreseeable or Xilinx had been advised of the possibility of the same. Xilinx assumes no obligation to correct any errors contained in the Materials or to notify you of updates to the Materials or to product specifications. You may not reproduce, modify, distribute, or publicly display the Materials without prior written consent. Certain products are subject to the terms and conditions of Xilinx's limited warranty, please refer to Xilinx's Terms of Sale which can be viewed at <http://www.xilinx.com/legal.htm#tos>; IP cores may be subject to warranty and support terms contained in a license issued to you by Xilinx. Xilinx products are not designed or intended to be fail-safe or for use in any application requiring fail-safe performance; you assume sole risk and liability for use of Xilinx products in such critical applications, please refer to Xilinx's Terms of Sale which can be viewed at <http://www.xilinx.com/legal.htm#tos>.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

© Copyright 2015-2016 Xilinx, Inc. Xilinx, the Xilinx logo, Artix, ISE, Kintex, Spartan, Virtex, Vivado, Zynq, and other designated brands included herein are trademarks of Xilinx in the United States and other countries. All other trademarks are the property of their respective owners.